

LAPORAN PROJECT
SISTEM MANAJEMEN STOK TOKO ELEKTRONIK
(Electronics Store Inventory System)

Disusun guna memenuhi tugas mata kuliah
ALGORITMA & STRUKTUR DATA

Dosen Pengampu :

Taufik Ridwan, M.T



Disusun Oleh

Kelompok 7 :

- | | |
|---|------------------------|
| 1. Muhammad Fadllan Ziadh Ramadhan | (2010631250066) |
| 2. Auzila Chantika Ardiana | (2510631250054) |
| 3. Muhamad Edward Rafael | (2510631250069) |

PROGRAM STUDI SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS SINGAPERBANGSA KARAWANG

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan project yang berjudul “**Sistem Manajemen Stok Toko Elektronik (Electronics Store Inventory System)**” dapat diselesaikan dengan baik.

Laporan ini disusun sebagai bentuk dokumentasi dan penjelasan mengenai proses perancangan serta pembuatan sistem manajemen stok yang bertujuan untuk membantu pengelolaan inventaris pada toko elektronik agar lebih efektif, efisien, dan terorganisir. Sistem ini dibuat untuk mempermudah proses pencatatan barang masuk dan keluar, pencarian data barang, pengelolaan stok, serta penyajian informasi inventaris secara lebih akurat.

Dalam penyusunan laporan ini, penulis menyadari bahwa masih terdapat kekurangan baik dari segi penulisan maupun isi laporan. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan dan pengembangan project di masa mendatang.

Penulis juga mengucapkan terima kasih kepada semua pihak yang telah membantu dan memberikan dukungan selama proses penyusunan project dan laporan ini berlangsung. Akhir kata, penulis berharap laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca mengenai penerapan sistem informasi dalam pengelolaan stok barang pada toko elektronik.

Karawang, 28 Mei 2026

Kelompok 7

DAFTAR ISI

KATA PENGANTAR	1
DAFTAR ISI	2
BAB I PENDAHULUAN	1
1.1. Latar Belakang	1
1.2. Tujuan Penulisan	1
BAB II ANALISIS KEBUTUHAN	3
2.1 Fitur – Fitur	3
2.2 Struktur Data	4
BAB III IMPLEMENTASI	5
3.1. Kode dan penjelasan.....	5
BAB IV ANALISIS KOMPLEKSITAS	15
4.1 Tabel dan penjelasan	15
BAB V PENGUJIAN	16
5.1 Output dalam berbagai scenario	16
BAB VI KESIMPULAN	22

BAB I

PENDAHULUAN

1.1. Latar Belakang

Perkembangan teknologi dan meningkatnya kebutuhan masyarakat terhadap perangkat elektronik membuat persaingan usaha toko elektronik semakin ketat. Toko elektronik tidak hanya dituntut menyediakan produk yang lengkap, tetapi juga harus mampu mengelola persediaan barang secara efektif dan efisien. Pengelolaan stok barang yang masih dilakukan secara manual sering menimbulkan berbagai permasalahan, seperti kesalahan pencatatan, keterlambatan pembaruan data, kehilangan informasi barang, serta ketidaksesuaian antara stok fisik dan data yang tercatat.

Selain itu, banyaknya jenis produk elektronik seperti televisi, laptop, smartphone, aksesoris, dan perangkat rumah tangga elektronik menyebabkan proses pengelolaan inventaris menjadi lebih kompleks. Jika tidak dikelola dengan baik, toko dapat mengalami kekurangan stok (out of stock) atau penumpukan barang yang dapat menghambat operasional dan menurunkan keuntungan usaha.

Oleh karena itu, diperlukan sebuah Sistem Manajemen Stok Toko Elektronik (Electronics Store Inventory System) yang dapat membantu proses pencatatan, pengelolaan, dan pemantauan stok barang secara terkomputerisasi. Sistem ini diharapkan mampu meningkatkan akurasi data, mempercepat proses pengelolaan inventaris, serta membantu pemilik toko dalam mengambil keputusan berdasarkan informasi stok yang tersedia secara real-time.

1.2. Tujuan Penulisan

- Mempermudah proses pencatatan barang masuk dan barang keluar sehingga mengurangi kesalahan pencatatan manual.
- Membantu pemilik toko memantau jumlah stok barang secara real-time.
- Meningkatkan efisiensi dan efektivitas pengelolaan inventaris pada toko elektronik.
- Menyediakan laporan data stok barang yang akurat dan mudah dipahami.

- Mengurangi risiko kehilangan data dan ketidaksesuaian antara stok fisik dengan data sistem.
- Membantu proses pengambilan keputusan terkait pengadaan dan penjualan barang elektronik.

BAB II

ANALISIS KEBUTUHAN

2.1 Fitur – Fitur

1. CRUD Operations :

- Tambah data barang baru ke
- Tampilkan Semua data barang
- Edit data berdasarkan id/kode barang
- Hapus data barang (soft delete dengan status)

2. Searching Features:

- Cari berdasarkan nama barang (Linear Search)
- Cari berdasarkan id/kode barang (Binary Search data sudah terurut)
- Cari berdasarkan kategori barang (menampilkan semua data dalam kategori tertentu)

3. Sorting Features:

- Urutkan berdasarkan id/kode barang (Ascending - Bubble Sort)
- Urutkan berdasarkan nama barang (Alphabetical - Selection Sort)
- Urutkan berdasarkan jumlah terbanyak stok barang (Descending)

4. Update status dan counter

Fitur ini untuk mengubah atau memperbarui kondisi/status suatu data dan penghitung otomatis barang dalam sistem.

5. Statistik Data (Total, Tersedia)

Fitur ini untuk menampilkan ringkasan informasi stok barang dalam sistem.

6. Simpan/Load Data ke/dari file teks

Fitur ini digunakan agar data tidak hilang ketika program ditutup.

7. Output dalam berbentuk file

Fitur ini untuk membuat output yang kita ingin kan dalam berbentuk file agar memudahkan dalam mengambil Keputusan.

2.2 Struktur Data

Program ini menggunakan *ArrayList<Barang>* dari package *java.util* sebagai struktur data utama untuk menyimpan koleksi objek barang. *ArrayList* dipilih karena ukurannya dapat menyesuaikan secara otomatis saat data ditambahkan atau dihapus, berbeda dengan array biasa yang ukurannya statis. Operasi CRUD pada data barang diimplementasikan melalui method yang dibuat secara manual, yaitu untuk menambahkan, menampilkan, mengubah, dan menghapus data barang dari list

BAB III

IMPLEMENTASI

3.1. Kode dan penjelasan

Untuk menjamin persistence data (data tidak hilang saat program dimatikan), program memanfaatkan pustaka *Google Gson* untuk konversi:

- Serialization (Save):

```
public static void save(List<Barang> barang) throws
IOException{
    try (Writer writer = new FileWriter(filePath)) {
        gson.toJson(barang, writer);
    }
}
```

Mengubah objek `List<Barang>` di dalam memori Java menjadi representasi format string terstruktur JSON yang ditulis langsung ke berkas fisik `src/data/data.json` menggunakan `FileWriter`.

- Deserialization(Load):

```
public static List<Barang> load() throws Exception {
    File file = new File(filePath);
    if (!file.exists()) {
        return new ArrayList<>();
    }

    try (Reader reader = new FileReader(file)) {
        Type type = new TypeToken<List<Barang>>() {
        }.getType();
        List<Barang> data = gson.fromJson(reader, type);

        return data != null ? data : new ArrayList<>();
    }
}
```

Membaca berkas fisik `data.json` dengan `FileReader` dan memetakannya kembali menjadi objek ber-tipe data Java generic `List<Barang>` melalui bantuan token refleksi `TypeToken`.

add(Barang barang)

```
public static void add(Barang barang) throws Exception {
    for (Barang b : list) {
        if (b.nama.equalsIgnoreCase(barang.nama)) {
            System.out.println("Barang sudah ada");
            return;
        }
    }
    list.add(barang);
    save(list);
}
```

Parameter: Barang barang. Return: void (Static). Menambahkan barang baru ke dalam database inventaris. Melakukan pengecekan terlebih dahulu agar tidak ada barang dengan nama yang sama (duplikat). Jika nama unik, barang dimasukkan ke list dan langsung disimpan ke file JSON.

delete(String key)

```
public static void delete(String key) throws Exception {
    Barang b = linearSearchNama(key);
    if (b != null) {
        b.status = false;
        save(list);
    }
}
```

Parameter: String key (nama barang yang dicari).

Return: void (Static).

Menerapkan fitur 'Soft Delete'. Mencari barang berdasarkan nama, apabila ditemukan maka atribut `status` barang diubah menjadi `false` (habis/terhapus) tanpa menghapus objeknya dari array. Dilanjutkan penyimpanan file.

```
Edit (Barang b, String inputNama, String inputJumlah,  
String inputKategori, String status)
```

```
public static void edit(Barang b, String inputNama, String  
inputJumlah, String inputKategori, String status) throws  
Exception {  
  
    b.nama = inputNama;  
  
    if (inputKategori != null && !inputKategori.isEmpty()) {  
  
        kategoriBarang katBaru = null;  
  
        try {  
  
            katBaru = kategoriBarang.valueOf(inputKategori);  
  
        } catch (IllegalArgumentException e) {  
  
        }  
  
        if (katBaru != null) {  
  
            b.kategori = katBaru;  
  
        }  
  
    }  
  
    if (!inputJumlah.isEmpty()) {  
  
        try {  
  
            b.stok = Integer.parseInt(inputJumlah);  
  
        } catch (NumberFormatException e) {  
  
        }  
  
    }  
  
    if (!status.isEmpty()) {  
  
        b.status = Boolean.parseBoolean(status);  
  
    }  
  
}
```

```

        try {

            save(list);

            System.out.println("\n[!] Data berhasil diupdate.");

        } catch (Exception e) {

            System.out.println("[!] Gagal update");

        }

    }
}

```

Parameter: Barang b, String inputNama, String inputJumlah, String inputKategori, String status.

Return: void (Static).

Memperbarui properti barang 'b' secara selektif. Memproses perubahan nama. Kategori dan stok diperbarui hanya jika string parameter masukan tidak kosong dan valid secara format numerik. Menyimpan data baru dan menampilkan tabel hasil.

linearSearchNama()

```

public static Barang linearSearchNama(String keyword) {

    for (Barang b : list) {

        if (b.nama.equalsIgnoreCase(keyword)) {

            return b;

        }

    }

    return null;

}

```

Parameter: String keyword (nama barang yang dicari).

Return: Barang (Static).

Melakukan pencarian barang berdasarkan nama secara sequential dari indeks pertama sampai terakhir dengan metode case-insensitive. Cara Kerja: Melakukan iterasi dari awal hingga akhir elemen dalam list untuk mencocokkan atribut `nama` dengan kata kunci pencarian (case-insensitive)

`binarySearchId(int idCari)`

```
public static Barang binarySearchId(int idCari) {  
    // Sort  
    SortByIdASC();  
  
    int kiri = 0;  
    int kanan = list.size() - 1;  
  
    while (kiri <= kanan) {  
        int tengah = (kiri + kanan) / 2;  
  
        if (list.get(tengah).id == idCari) {  
            return list.get(tengah);  
        } else if (list.get(tengah).id < idCari) {  
            kiri = tengah + 1;  
        } else {  
            kanan = tengah - 1;  
        }  
    }  
  
    return null;  
}
```

- Parameter: int idCari.

- Return: Barang (Static).

Melakukan pencarian barang berbasis ID secara cepat dengan algoritma *Divide and Conquer* (pembagian dua ruang). Terlebih dahulu mengurutkan list menggunakan Bubble Sort agar list berada dalam keadaan terurut menaik (syarat biner). Cara Kerjanya Algoritma pencarian efisien bertipe **Divide and Conquer**. Bekerja dengan cara membagi dua ruang pencarian secara berulang. Syarat mutlak: data harus dalam kondisi terurut. Oleh sebab itu, di dalam metode `binarySearchId`,

program secara otomatis memanggil algoritma `SortByIdASC()` terlebih dahulu sebelum proses pencarian dimulai.

`selectionSortByName()`

```
public static void selectionSortByName() {
    int n = list.size();

    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (list.get(j).nama.compareToIgnoreCase(
                list.get(minIndex).nama) < 0) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            Barang temp = list.get(i);
            list.set(i, list.get(minIndex));
            list.set(minIndex, temp);
        }
    }
}
```

- Return: void (Static).

Cara Kerjanya dengan membagi array menjadi bagian terurut dan tidak terurut. Secara berulang mencari elemen terkecil dari bagian yang belum terurut dan menukarnya ke posisi awal bagian belum terurut tersebut.

Kompleksitas Waktu:

- Best/Worst Case: $O(n^2)$ - Algoritma ini selalu memindai sisa list untuk mencari elemen minimum tanpa peduli apakah data sudah terurut sebagian.

SortByIdASC()

```
public static void SortByIdASC() {
    int n = list.size();
    boolean swapped;
    for (int i = 0; i < n - 1; i++) {
        swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (list.get(j).id > list.get(j + 1).id) {
                Barang temp = list.get(j);
                list.set(j, list.get(j + 1));
                list.set(j + 1, temp);
                swapped = true;
            }
        }
        // Optimasi
        if (!swapped) break; } }
```

- Return: void (Static).

Mengurutkan elemen di dalam list berdasarkan ID terkecil ke terbesar menggunakan algoritma Bubble Sort in-place disertai optimasi bendera swap. Dengan membandingkan elemen-elemen yang berdekatan dan menukarnya jika berada dalam urutan yang salah. Proses ini diulang terus-menerus sampai seluruh list terurut. Untuk optimasi Terdapat flag boolean `swapped`. Jika dalam satu putaran penuh (pass) tidak terjadi penukaran sama sekali, itu berarti data sudah terurut, sehingga loop dihentikan lebih cepat.

Kompleksitas Waktu:

- Best Case: $O(n)$ - ketika list sudah terurut dari awal (hanya butuh 1 pass).
- Worst Case: $O(n^2)$ - ketika list terurut terbalik (harus melakukan $n*(n-1)/2$ perbandingan dan pertukaran).

`insertionSortByStok()`

```
public static void insertionSortByStok() {
    int n = list.size();

    for (int i = 1; i < n; i++) {
        Barang key = list.get(i);
        int j = i - 1;
        while (j >= 0 && list.get(j).stok < key.stok) {
            list.set(j + 1, list.get(j));
            j--;
        }
        list.set(j + 1, key);
    }
}
```

- Return: void (Static).

Cara Kerjanya dengan mengambil elemen satu per satu, kemudian menyisipkannya pada posisi yang tepat di dalam sub-list yang sudah terurut sebelumnya.

Kompleksitas Waktu:

- Best Case: $O(n)$ - terjadi ketika list masukan sudah terurut menurun.
- Worst Case: $O(n^2)$ - terjadi ketika list masukan terurut menaik (setiap elemen baru harus digeser hingga awal sub-list).

`exportBarang(List<Barang> daftarBarang)`

```
for (Barang barang : daftarBarang) {

    table.addCell(new PdfPCell(new Paragraph(String.valueOf(
        barang.getId()), biasa)));

    table.addCell(new PdfPCell(new Paragraph(barang.getNama(
        ), biasa)));
    // DAN ATTRIBUTE DATA LAINYA..
    document.add(table);
    document.close();
}
}
```

- Parameter: List<Barang> daftarBarang.

- Return: void.

Membuat dokumen PDF baru bernama "*laporan_barang_[tanggal_sekarang].pdf*" di dalam folder "*reports/*". Memformat halaman, menulis judul laporan, menyisipkan tabel data iText PDF, mengatur rasio lebar kolom, merender isi daftar barang ke sel-sel tabel, dan menyimpan file tersebut.

start(Stage stage)

```
@Override
public void start(Stage stage) throws Exception {
    // exeption jafafx handler. biar apl ny jalan tanpa error
    karna bug bawaan jafafx
    Thread.currentThread().setUncaughtExceptionHandler((t, e) -> {
//        //ngambil thread dan exeption
        if (e instanceof IndexOutOfBoundsException // cek exception
            && e.getStackTrace().length > 0
            && (e.getStackTrace()[0].getClassName().contains
                ("ReadOnlyUnbackedObservableList"
                || e.getStackTrace()[0].getClassName().contains
                    ("SelectedItemsReadOnlyObservableList"))) {
            // bug JavaFX 26, diabaikan
            return;
        }
        // print exeption lain
        e.printStackTrace();
    });

    FXMLLoader loader = new FXMLLoader(
        getClass().getResource("/gui/Main.fxml")
    );
    BorderPane root = loader.load();
    Scene scene = new Scene(root, 900, 600);
    stage.setTitle("Inventaris");
    stage.setScene(scene);
    stage.show();
}
```

- Return: void.

- Deskripsi: Entry point aplikasi visual GUI JavaFX. Memasang event handler error global kustom untuk menangkap bug internal platform JavaFX SDK, memuat file desain tata letak FXML ("*Main.fxml*"), merakitnya ke kontainer BorderPane dan Scene, serta menampilkan Window aplikasi ke monitor. terdapat bug thread bawaan platform di mana interaksi cepat dengan elemen ListView / ComboBox terkadang memicu *IndexOutOfBoundsException* di class tertutup JDK (*SelectedItemsReadOnlyObservableList*). Tanpa handler ini, thread JavaFX akan

crash seketika dan menutup paksa aplikasi (force close). Dengan menangkap exception spesifik tersebut dan mengabaikannya secara aman, program tetap stabil dan terus berjalan dengan performa penuh.

`main(String[] args)`

- Parameter: `String[] args`.
- Return: `void`.
- Deskripsi: Titik awal jalannya seluruh program Java. Melakukan inisialisasi awal dengan memuat berkas database JSON, menembak instruksi '`launch(args)`' untuk memulai sistem GUI JavaFX, dan menampung kode perulangan CLI (menu interaktif) jika GUI ditutup oleh pengguna.

BAB IV

ANALISIS KOMPLEKSITAS

4.1 Tabel dan penjelasan

No	Fitur / Proses	Algoritma	Kompleksitas Waktu	Penjelasan
1	Menampilkan Barang	Traversal ArrayList	$O(n)$	Program membaca seluruh data barang satu per satu
2	Tambah Barang	Insert ArrayList	$O(1)$	Data ditambahkan di akhir list
3	Hapus Barang	Linear Search	$O(n)$	Program mencari nama barang terlebih dahulu
4	Update Barang	Linear Search	$O(n)$	Program mencari ID barang sebelum diubah
5	Cari Nama Barang	Linear Search	$O(n)$	Data dicek satu per satu hingga ditemukan
6	Cari ID Barang	Binary Search	$O(\log n)$	Data dibagi menjadi dua bagian setiap proses pencarian
7	Sorting Data ID	Sorting	$O(n \log n)$	Digunakan sebelum Binary Search
8	Save JSON	Write File	$O(n)$	Semua data ditulis ke file JSON
9	Load JSON	Read File	$O(n)$	Semua data dibaca dari file JSON

BAB V

PENGUJIAN

5.1 Output dalam berbagai scenario

1. Menambahkan produk baru

Tambah Barang

Form Tambah

test

1231

Tambah

ID	Nama Barang	Kategori	Stok
2345	test edit	rumahTangga	23 pcs
2346	Keyboard Mechanical...	komputer	10 pcs
3452	Monitor Gaming 144Hz	komputer	10 pcs
4716	Mouse Wireless Logit...	komputer	10 pcs
8723	SSD NVMe 512GB	komputer	10 pcs
12345	Laptop ASUS Zenbook	komputer	5 pcs
12346	MacBook Air M2	komputer	3 pcs

Tambah Barang

Form Tambah

Sukses

Data berhasil ditambahkan.

OK

Tambah

ID	Nama Barang	Kategori	Stok
2345	test edit	rumahTangga	23 pcs
2346	Keyboard Mechanical...	komputer	10 pcs
3452	Monitor Gaming 144Hz	komputer	10 pcs
4716	Mouse Wireless Logit...	komputer	10 pcs
8723	SSD NVMe 512GB	komputer	10 pcs
12345	Laptop ASUS Zenbook	komputer	5 pcs
12346	MacBook Air M2	komputer	3 pcs

2. Edit data produk

Inventaris

kelompok 7

MAIN

Home

KELOLA

Tambah Barang

Edit Barang

Hapus Barang

Edit Barang

Form Edit

ID Barang

Nama barang

Stok barang

Status barang

Submit

ID	Nama Barang	Kategori	Stok
2345	test ganti	komputer	29 pcs
2346	Keyboard Mechanical	komputer	10 pcs
3452	Monitor Gaming 144Hz	komputer	10 pcs
4716	Mouse Wireless Logitech	komputer	10 pcs
8723	SSD NVMe 512GB	komputer	10 pcs
12345	Laptop ASUS Zenbook	komputer	5 pcs
12346	MacBook Air M2	komputer	3 pcs

Inventaris

kelompok 7

MAIN

Home

KELOLA

Tambah

Edit Bara

Hapus B

Edit Barang

Form Edit

true

Submit

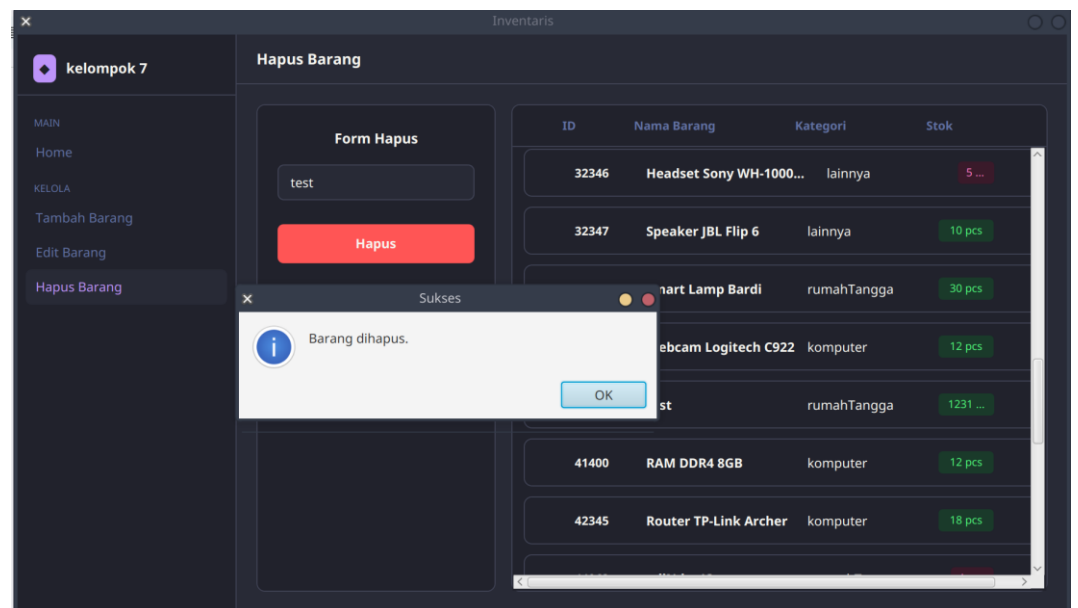
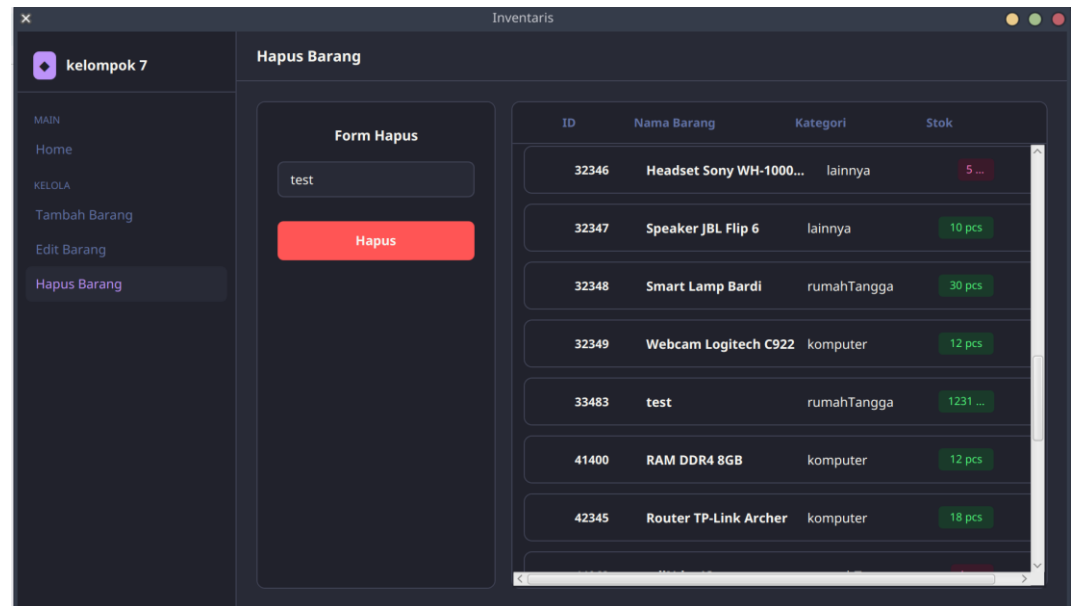
Sukses

Data berhasil diupdate.

OK

ID	Nama Barang	Kategori	Stok
2345	test edit	rumahTangga	23 pcs
2346	Keyboard Mechanical	komputer	10 pcs
3452	Monitor Gaming 144Hz	komputer	10 pcs
4716	Mouse Wireless Logitech	komputer	10 pcs
8723	SSD NVMe 512GB	komputer	10 pcs
12345	Laptop ASUS Zenbook	komputer	5 pcs
12346	MacBook Air M2	komputer	3 pcs

3. Hapus produk



4. Sort by Id (Ascending)

The screenshot shows the 'Inventaris' application interface. On the left is a sidebar with a user profile 'kelompok 7' and navigation links: 'Home' (under MAIN), 'Tambah Barang', 'Edit Barang', and 'Hapus Barang' (under KELOLA). The main area displays summary statistics: 'TOTAL BARANG: 32', 'TOTAL STOK: 459', and 'DIHAPUS: 4'. Below these are filters: 'ascending' (selected), 'Nama', 'Kategori', and 'Stok'. The table lists items sorted by ID in ascending order.

ID	Nama	Kategori	Stok
2345	test ganti	komputer	29 pcs
2346	Keyboard Mechanical RGB	komputer	10 pcs
3452	Monitor Gaming 144Hz	komputer	10 pcs
4716	Mouse Wireless Logitech	komputer	10 pcs
8723	SSD NVMe 512GB	komputer	10 pcs

5. Sort by name

The screenshot shows the 'Inventaris' application interface with the same sidebar as above. The main area summary statistics remain the same. The filters are now: 'ID', 'A-Z' (selected), 'Kategori', and 'Stok'. The table lists items sorted by name in ascending order.

ID	Nama	Kategori	Stok
22349	Air Fryer Xiaomi	rumahTangga	7 pcs
22348	Blender Philips HR2115	rumahTangga	12 pcs
51724	Harddisk Eksternal 1TB	komputer	10 pcs
32346	Headset Sony WH-1000XM5	lainnya	5 pcs
71243	Kabel LAN Cat6 10m	komputer	10 pcs

6. Sort by category

The screenshot shows the 'Inventaris' application interface. On the left is a sidebar with a user profile 'kelompok 7' and a menu with options: Home, Tambah Barang, Edit Barang, and Hapus Barang. The main area displays summary statistics: TOTAL BARANG (32), TOTAL STOK (459), and DIHAPUS (4). Below these are filters for ID, Nama, and Stok, with 'komputer' selected in the category dropdown. A table lists items, all of which are in the 'komputer' category. The items are sorted by their ID in ascending order.

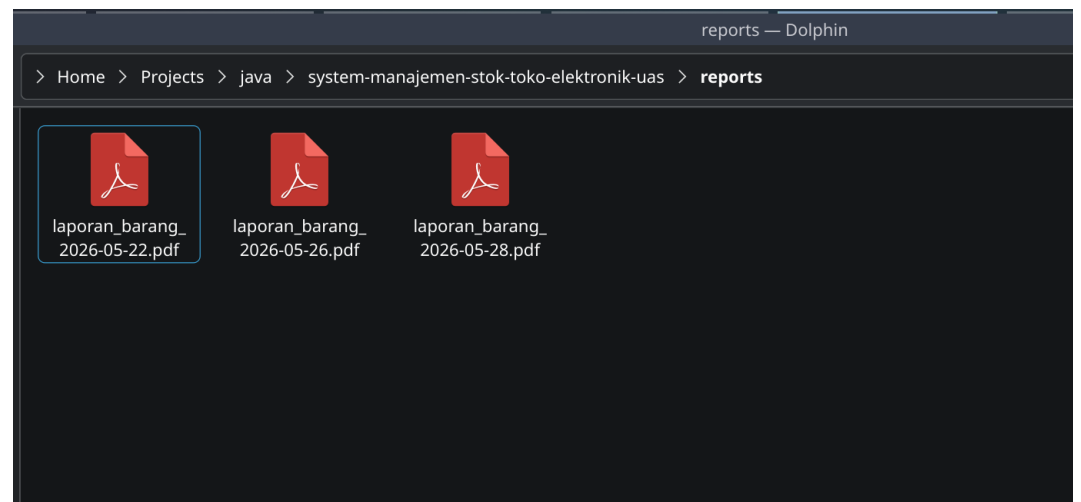
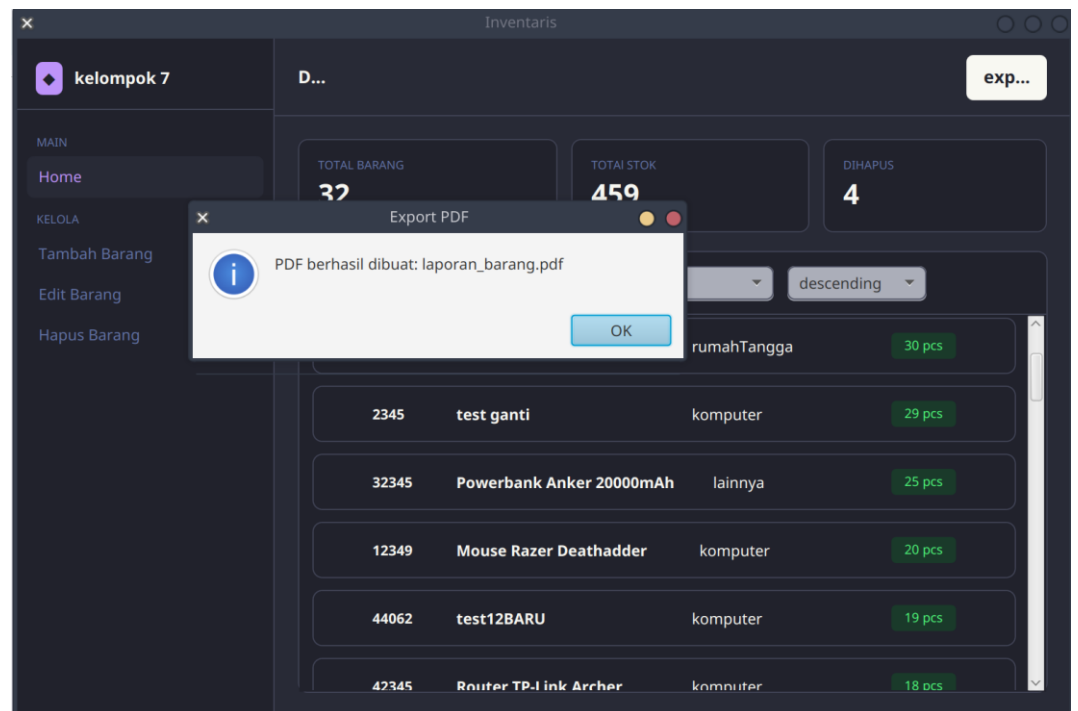
ID	Nama	komputer	Stok
3452	Monitor Gaming 144Hz	komputer	10 pcs
4716	Mouse Wireless Logitech	komputer	10 pcs
8723	SSD NVMe 512GB	komputer	10 pcs
12345	Laptop ASUS Zenbook	komputer	5 pcs
12346	MacBook Air M2	komputer	3 pcs
12347	Monitor LG 24 Inch	komputer	10 pcs

7. Sort by stok (descending)

The screenshot shows the 'Inventaris' application interface with the same sidebar. The main area displays the same summary statistics. The filters show 'Kategori' selected with 'descending' chosen in the sort dropdown. The table lists items sorted by their stock in descending order. The items belong to various categories: rumahTangga, komputer, and lainnya.

ID	Nama	Kategori	descending
32348	Smart Lamp Bardi	rumahTangga	30 pcs
2345	test ganti	komputer	29 pcs
32345	Powerbank Anker 20000mAh	lainnya	25 pcs
12349	Mouse Razer Deathadder	komputer	20 pcs
44062	test12BARU	komputer	19 pcs
42345	Router TP-Link Archer	komputer	18 pcs

8. Export data to pdf



BAB VI

KESIMPULAN

Berdasarkan hasil perancangan dan implementasi Sistem Manajemen Stok Toko Elektronik, dapat disimpulkan bahwa sistem ini berhasil membantu proses pengelolaan data barang secara lebih terstruktur, cepat, dan efisien. Sistem mampu mendukung pengelolaan stok barang elektronik seperti penambahan data produk, pembaruan stok, pencarian barang, serta penghapusan data produk dengan lebih mudah dibandingkan pencatatan manual.

Penggunaan bahasa pemrograman Java dengan konsep Object-Oriented Programming (OOP) membuat sistem memiliki struktur kode yang lebih rapi, modular, dan mudah dikembangkan kembali di masa mendatang. Selain itu, penggunaan format JSON sebagai media penyimpanan data memberikan kemudahan dalam proses membaca dan menyimpan data, sedangkan library Gson membantu proses parsing data JSON menjadi lebih sederhana dan efisien.

Dengan adanya sistem ini, proses manajemen stok pada toko elektronik dapat dilakukan dengan lebih akurat sehingga dapat meminimalkan kesalahan pencatatan data barang. Project ini juga menjadi sarana pembelajaran dalam penerapan konsep pemrograman berorientasi objek, pengelolaan data, penggunaan dependency management menggunakan Maven, serta implementasi penyimpanan data berbasis JSON dalam pengembangan aplikasi Java.